

ডিসিশন ট্রি (Decision Tree) অ্যালগরিদম

মেশিন লার্নিং নোটস — সহজ বাংলায় ব্যাখ্যা

১. ডিসিশন ট্রি কী?

ডিসিশন ট্রি হলো মেশিন লার্নিং-এর একটি সুপারভাইজড লার্নিং (Supervised Learning) অ্যালগরিদম, যা ক্লাসিফিকেশন (Classification) এবং রিগ্রেশন (Regression)—দুই ধরনের সমস্যাতেই ব্যবহার করা যায়। নাম শুনেই বোঝা যায়, এটি দেখতে অনেকটা একটি গাছের (Tree) মতো, যেখানে প্রতিটি সিদ্ধান্ত (Decision) একটি শাখা তৈরি করে।

সহজ ভাষায় বলতে গেলে, ডিসিশন ট্রি এমনভাবে কাজ করে যেভাবে আমরা মানুষ প্রতিদিন সিদ্ধান্ত নিই। যেমন — "আজ কি বৃষ্টি হবে?" এই প্রশ্নের উত্তর খুঁজতে আমরা প্রথমে দেখি আকাশ মেঘলা কিনা, তারপর বাতাসে অর্ধতা কেমন, তারপর তাপমাত্রা কেমন — এভাবে ধাপে ধাপে প্রশ্ন করে করে একটি সিদ্ধান্তে পৌঁছাই। ডিসিশন ট্রিও ঠিক একইভাবে ডেটাকে বিভিন্ন প্রশ্নের মাধ্যমে ভাগ করতে করতে একটি চূড়ান্ত সিদ্ধান্তে (prediction) পৌঁছায়।

২. ডিসিশন ট্রি-র গঠন (Structure)

একটি ডিসিশন ট্রি সাধারণত নিচের অংশগুলো নিয়ে গঠিত:

- রুট নোড (Root Node):** ট্রি-র একদম শুরুর নোড, যেখানে পুরো ডেটাসেট থাকে এবং প্রথম প্রশ্নটি করা হয়।
- ইন্টারনাল নোড / ডিসিশন নোড (Decision Node):** এখানে একটি ফিচার (feature) বা বৈশিষ্ট্য নিয়ে প্রশ্ন করা হয়, এবং সেই প্রশ্নের উত্তরের ভিত্তিতে ডেটা আরও ভাগ হয়।
- ব্রাঞ্চ / এজ (Branch):** একটি নোড থেকে অন্য নোডে যাওয়ার পথ, যা প্রশ্নের সম্ভাব্য উত্তর নির্দেশ করে (যেমন হ্যাঁ/না)।
- লিফ নোড (Leaf Node):** ট্রি-র শেষ প্রান্তের নোড, যেখানে আর কোনো ভাগ হয় না এবং এখানেই চূড়ান্ত ফলাফল বা প্রেডিকশন (prediction) পাওয়া যায়।

উদাহরণ: ধরা যাক আমরা একটি ডিসিশন ট্রি বানাচ্ছি যা দিয়ে বোঝা যাবে কেউ বাইরে খেলতে যাবে কিনা।

রুট নোড: "আবহাওয়া কেমন?" (রোদ / মেঘলা / বৃষ্টি)

→ যদি "রোদ" হয়, পরের প্রশ্ন: "অর্ধতা কি বেশি?"

→ যদি "বৃষ্টি" হয়, পরের প্রশ্ন: "বাতাস কি জোরে বইছে?"

→ প্রতিটি শাখার শেষে একটি লিফ নোড থাকবে যা বলবে "হ্যাঁ, খেলতে যাবে" অথবা "না, যাবে না"।

৩. ডিসিশন ট্রি কীভাবে কাজ করে?

ডিসিশন ট্রি তৈরি হয় রিকার্সিভ পার্টিশনিং (Recursive Partitioning) পদ্ধতিতে — অর্থাৎ ডেটাকে বারবার ছোট ছোট ভাগে ভাগ করা হয়, যতক্ষণ না প্রতিটি ভাগ (বা লিফ নোড) মোটামুটি "বিশুদ্ধ" (pure) হয়ে যায়, মানে সেই ভাগে বেশিরভাগ ডেটা একই ক্লাসের হয়।

ধাপগুলো:

- পুরো ডেটাসেট নিয়ে শুরু করা হয় (রুট নোড)।
- সবচেয়ে ভালো ফিচার (best feature) খুঁজে বের করা হয়, যেটি দিয়ে ডেটাকে ভাগ করলে সবচেয়ে ভালো আলাদা (separation) করা যায়।
- সেই ফিচারের ভিত্তিতে ডেটাকে দুই বা ততোধিক ভাগে ভাগ করা হয় (branch তৈরি হয়)।
- প্রতিটি ভাগের জন্য একই প্রক্রিয়া আবার পুনরাবৃত্তি (repeat) করা হয়।
- যখন কোনো একটি ভাগে আর ভাগ করার দরকার হয় না (যেমন সব ডেটা একই ক্লাসের, বা নির্দিষ্ট গভীরতায় পৌঁছে গেছে), তখন সেটিকে লিফ নোড বানিয়ে ট্রি তৈরি বন্ধ করা হয়।

সবচেয়ে ভালো ফিচার কীভাবে বাছাই করা হয়?

এখানেই আসল প্রশ্ন — কোন ফিচার দিয়ে ভাগ করলে সবচেয়ে ভালো ফলাফল পাওয়া যাবে? এটি নির্ধারণ করার জন্য কয়েকটি গাণিতিক মাপকাঠি (metric) ব্যবহার করা হয়:

ক) এনট্রপি (Entropy) ও ইনফরমেশন গেইন (Information Gain)

এনট্রপি হলো একটি ডেটাসেটের "অগোছালো ভাব" বা অনিশ্চয়তা (impurity/randomness) মাপার একটি পদ্ধতি। যদি একটি নোডে সব ডেটা একই ক্লাসের হয়, তাহলে এনট্রপি হবে ০ (সম্পূর্ণ বিশুদ্ধ)। আর যদি ডেটা সমান সমান দুই ক্লাসে ভাগ হয়ে থাকে, তাহলে এনট্রপি হবে সর্বোচ্চ।

$$\text{এনট্রপি}(S) = - \sum p_i \times \log_2(p_i)$$

এখানে p_i হলো i -তম ক্লাসের ডেটার অনুপাত। কোনো ফিচার দিয়ে ভাগ করার পর এনট্রপি যতটা কমে, তাকেই বলা হয় ইনফরমেশন গেইন। যে ফিচার সবচেয়ে বেশি ইনফরমেশন গেইন দেয়, সেই ফিচারকেই ভাগ করার জন্য বেছে নেওয়া হয়।

খ) জিনি ইনডেক্স (Gini Index)

জিনি ইনডেক্সও এনট্রপির মতোই একটি "অবিশুদ্ধতা" (impurity) মাপার পদ্ধতি, তবে এটি গণনা করা তুলনামূলক সহজ ও দ্রুত। CART (Classification and Regression Tree) অ্যালগরিদমে এটি বেশি ব্যবহৃত হয়।

$$জিনি(S) = 1 - \sum (p_i)^2$$

জিনি ইনডেক্সের মান যত কম, নোডটি তত বেশি বিশুদ্ধ। যে ফিচার ভাগ করার পর জিনি ইনডেক্স সবচেয়ে কম হয়, সেই ফিচারকেই বেছে নেওয়া হয়।

গ) রিগ্রেশনের ক্ষেত্রে (Variance Reduction)

যখন ডিসিশন ট্রি সংখ্যাগত মান (continuous value) প্রেডিক্ট করে (রিগ্রেশন সমস্যা), তখন ক্লাস পিউরিটির বদলে ভ্যারিয়েন্স (Variance) বা স্ট্যান্ডার্ড ডেভিয়েশন কমানোর দিকে লক্ষ্য রাখা হয়। যে ভাগে ভ্যারিয়েন্স সবচেয়ে বেশি কমে, সেই ভাগটিই বেছে নেওয়া হয়।

৪. ওভারফিটিং সমস্যা ও প্রুনিং (Pruning)

ডিসিশন ট্রি যদি খুব গভীরভাবে (deep) বাড়তে থাকে, তাহলে এটি ট্রেনিং ডেটার প্রতিটি খুঁটিনাটি (noise) পর্যন্ত মুখস্থ করে ফেলে। একে বলা হয় ওভারফিটিং (Overfitting) — অর্থাৎ ট্রেনিং ডেটায় খুব ভালো ফলাফল দিলেও নতুন ডেটায় (test data) খারাপ ফলাফল দেয়।

এই সমস্যা সমাধানের জন্য প্রুনিং (Pruning) নামক পদ্ধতি ব্যবহার করা হয়, যার মাধ্যমে ট্রি-র অপ্রয়োজনীয় শাখাগুলো ছাঁটাই করা হয়:

- প্রি-প্রুনিং (Pre-pruning):** ট্রি তৈরির সময়ই কিছু শর্ত বেঁধে দেওয়া হয় — যেমন সর্বোচ্চ গভীরতা (max_depth), একটি নোডে সর্বনিম্ন কতগুলো ডেটা থাকতে হবে (min_samples_split) ইত্যাদি, যাতে ট্রি প্রয়োজনের বেশি বড় না হয়।
- পোস্ট-প্রুনিং (Post-pruning):** প্রথমে সম্পূর্ণ ট্রি তৈরি করা হয়, তারপর যেসব শাখা মডেলের performance-এ তেমন প্রভাব ফেলে না, সেগুলো পরে ছাঁটাই ফেলা হয়।

৫. ডিসিশন ট্রি-র সুবিধা (Advantages)

- ✓ বোঝা ও ব্যাখ্যা করা সহজ:
মডেলটি দেখতে গেলেই মতো হওয়ায় সাধারণ মানুষও এর সিদ্ধান্ত নেওয়ার প্রক্রিয়া সহজে বুঝতে পারে। একে "White Box Model" বলা হয়।
- ✓ ডেটা প্রি-প্রসেসিং কম লাগে:
ফিচার স্কেলিং (normalization/standardization) করার প্রয়োজন হয় না।
- ✓ সংখ্যাগত ও শ্রেণিবদ্ধ (numerical ও categorical) — দুই ধরনের ডেটাই ব্যবহার করা যায়।
- ✓ নন-লিনিয়ার সম্পর্ক বোঝা যায়:
ফিচারগুলোর মধ্যে জটিল ও নন-লিনিয়ার সম্পর্ক থাকলেও ডিসিশন ট্রি তা ধরতে পারে।
- ✓ ফিচার সিলেকশনে সাহায্য করে:
কোন ফিচারগুলো গুরুত্বপূর্ণ তা ট্রি নিজেই বের করে দেয় (feature importance)।

৬. ডিসিশন ট্রি-র অসুবিধা (Disadvantages)

- ✗ ওভারফিটিং-এর ঝুঁকি:
নিয়ন্ত্রণ না করলে ট্রি অতিরিক্ত জটিল হয়ে ট্রেনিং ডেটা মুখস্থ করে ফেলে।
- ✗ অস্থিতিশীলতা (Instability):
ডেটাতে সামান্য পরিবর্তন হলেও সম্পূর্ণ ভিন্ন গঠনের ট্রি তৈরি হতে পারে।
- ✗ পক্ষপাতিত্ব (Bias):
যেসব ফিচারে বেশি ক্যাটাগরি (levels) থাকে, ট্রি সেগুলোর দিকে বেশি ঝুঁকি পড়তে পারে।
- ✗ একক ট্রি সাধারণত কম নির্ভুল:
Random Forest বা Gradient Boosting-এর মতো এনসেম্বল (ensemble) পদ্ধতির তুলনায় একটি একক ডিসিশন ট্রি সাধারণত কম নির্ভুলতা (accuracy) দেয়।
- ✗ গ্রিডি অ্যালগরিদম:
প্রতিটি ধাপে স্থানীয়ভাবে (locally) সবচেয়ে ভালো সিদ্ধান্ত নেয়, যা সবসময় সামগ্রিকভাবে (globally) সেরা সমাধান নাও হতে পারে।

৭. জনপ্রিয় ডিসিশন ট্রি অ্যালগরিদম

অ্যালগরিদম	ব্যবহৃত মেট্রিক	বৈশিষ্ট্য
ID3	ইনফরমেশন গেইন	শুধু ক্যাটাগরিক্যাল ফিচার নিয়ে কাজ করে
C4.5	গেইন রেশিও	ID3-এর উন্নত সংস্করণ, continuous ডেটাও সাপোর্ট করে
CART	জিনি ইনডেক্স / ভ্যারিয়েন্স	ক্লাসিফিকেশন ও রিগ্রেশন উভয়ই করতে পারে; সবচেয়ে বেশি ব্যবহৃত (scikit-learn এ ব্যবহৃত)

৮. বাস্তব জীবনে ব্যবহার (Use Cases)

- মেডিকেল ডায়াগনসিস: রোগীর উপসর্গের ভিত্তিতে রোগ শনাক্ত করা।
- ব্যাংকিং ও ফ্রাড: লোন অ্যাপ্রোভাল করা উচিত কিনা, বা কোনো লেনদেন প্রতারণামূলক (fraud) কিনা তা যাচাই করা।

- কার্টমার চার্ন প্রতিক্রিয়া: কোন গ্রাহক সেবা ছেড়ে চলে যেতে পারে তা আগেই অনুমান করা।
- মার্কেটিং: কোন গ্রাহক কোন বিজ্ঞাপনে সাড়া দেবে তা নির্ধারণ করা।
- Random Forest** ও **Gradient Boosting**-এর ভিত্তি: আধুনিক অনেক শক্তিশালী মডেল (যেমন XGBoost) মূলত অনেকগুলো ডিসিশন ট্রি একসাথে ব্যবহার করে তৈরি হয়।

৯. সংক্ষিপ্ত সারাংশ (Summary)

- ডিসিশন ট্রি একটি সহজবোধ্য, গাছ-আকৃতির মডেল যা ধাপে ধাপে প্রশ্ন করে সিদ্ধান্তে পৌঁছায়।
- এনট্রপি, ইনফরমেশন গেইন, ও জিনি ইনডেক্স ব্যবহার করে সবচেয়ে ভালো ফিচার বাছাই করা হয়।
- ওভারফিটিং এড়াতে প্রনিং করা প্রয়োজন।
- এটি ব্যাখ্যাযোগ্য (interpretable) হলেও একক মডেল হিসেবে এর নির্ভুলতা সীমিত, তাই Random Forest-এর মতো এনসেম্বল মডেলে এটি বহুল ব্যবহৃত।